

Network Science: Exam Revision Notes

0 Contents

1	Graph Basics	2
1.1	Connected Components	2
1.2	Breadth-First Search (BFS)	2
2	Community Detection Algorithms	2
2.1	Agglomerative Hierarchical Clustering	2
2.2	Divisive Algorithm (Girvan–Newman)	3
2.3	Modularity-Based Greedy Algorithm (Newman)	3
3	Graph Partitioning	3
3.1	Kernighan–Lin Algorithm	3
4	Overlapping Communities	4
4.1	Clique Percolation Method (CPM)	4
4.2	Link Clustering	4
5	Network Models	4
5.1	Watts–Strogatz Model	4
5.2	Barabási–Albert (BA) Model	5
5.3	Linearized Chord Diagram (LCD) Model	5
6	Origin of Preferential Attachment	5
6.1	Link Selection Model	5
6.2	Copying Model	6
6.3	Optimization Model	6
6.4	Model A: Growth Only (No Preferential Attachment)	6
6.5	Model B: Preferential Attachment Only (No Growth)	6
7	Randomization / Null Models	6
7.1	Random Cuisine Strategy 1: Ingredient Randomization	6
7.2	Random Cuisine Strategy 2: Size-Preserving Randomization	7
8	Application Model	7
8.1	Bartoli’s Model for Protein Contact Maps	7
9	Configuration Model and Deg Preserving Ran	7
9.1	Degree-Preserving Randomization (Edge Rewiring)	7
9.2	Configuration Model	8

1 Graph Basics

1.1 Connected Components

A connected component is a maximal set of nodes such that every pair of nodes is reachable from one another via some path.

Algorithm:

1. Initialize all nodes as unvisited.
2. Set component label counter $c = 0$.
3. For each node v in the graph:
 - (a) If v is unvisited:
 - i. Increment $c \leftarrow c + 1$.
 - ii. Run BFS starting from v .
 - iii. Mark all nodes reachable from v with label c .
4. Continue until all nodes are visited.

Notes:

- Each BFS traversal discovers exactly one connected component.
- Works for undirected graphs.

1.2 Breadth-First Search (BFS)

Purpose: Compute shortest paths in unweighted graphs (distance = number of edges).

Algorithm:

1. *Input:* Graph G , source node s .
2. *Initialization:*
 - Set $d(s) = 0$.
 - Set $d(v) = \infty$ for all other nodes v .
 - Mark all nodes as unvisited.
3. Create a queue Q and insert s ; mark s visited.
4. While Q is not empty:
 - (a) Remove node u from the front of Q .
 - (b) For each neighbor v of u :
 - i. If v is unvisited:
 - Mark v as visited.
 - Set $d(v) = d(u) + 1$.
 - Insert v into Q .

Time complexity:

$$\mathcal{O}(N + L)$$

where N is the number of nodes and L is the number of edges.

2 Community Detection Algorithms

2.1 Agglomerative Hierarchical Clustering

Idea: Iteratively merge the most similar clusters, using a similarity matrix (e.g., topological overlap).

Algorithm:

1. Compute a pairwise similarity matrix for all node pairs.
2. Initialize: each node forms its own cluster.

3. Repeat:
 - (a) Find the pair of clusters with the highest similarity.
 - (b) Merge them into a single cluster.
 - (c) Update the similarity matrix using one of:
 - *Single linkage*: minimum pairwise distance between clusters.
 - *Complete linkage*: maximum pairwise distance between clusters.
 - *Average linkage*: average pairwise distance between clusters.
4. Continue until only one cluster remains.
5. Represent the merging process as a dendrogram.
6. Choose a cut level on the dendrogram to obtain the desired communities.

Time complexity: $\mathcal{O}(N^2)$ approximately.

2.2 Divisive Algorithm (Girvan–Newman)

Idea: Edges connecting different communities are traversed frequently and thus have high betweenness centrality.

Algorithm:

1. Compute edge betweenness centrality for all edges in the graph.
2. Remove the edge with the highest betweenness.
3. Recompute betweenness centrality for all remaining edges.
4. Repeat steps 2–3 until no edges remain.
5. Track connected components at each removal step.
6. Compute modularity Q for each resulting partition.
7. Select the partition with maximum modularity Q .

Note: Computationally expensive due to repeated recomputation of betweenness after each removal.

2.3 Modularity-Based Greedy Algorithm (Newman)

Modularity Q measures the density of edges within communities relative to what would be expected at random:

$$Q = \frac{1}{2L} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2L} \right) \delta(c_i, c_j)$$

where L is the total number of edges, k_i is the degree of node i , A_{ij} is the adjacency matrix, and $\delta(c_i, c_j) = 1$ if i and j are in the same community.

Algorithm:

1. Start with each node in its own community.
2. Compute the modularity gain ΔQ for merging each pair of adjacent communities.
3. Merge the pair with the highest ΔQ .
4. Update the modularity matrix.
5. Repeat steps 2–4 until all nodes are in one community.
6. Track modularity Q at each step.
7. Select the partition corresponding to maximum Q .

Note: Faster than Girvan–Newman; scales to larger networks.

3 Graph Partitioning

3.1 Kernighan–Lin Algorithm

Objective: Partition nodes into two equal-sized groups A and B to minimize the number of edges crossing between groups (edge cut).

Algorithm:

Goal: Split a network into 2 equal groups while reducing the connections btw them.

1. **Step 1:** Divide the graph into two equal groups (A & B)
2. **Step 2:** Evaluate swaps for every pair (one from each group), check how swapping affects cut size (inter links)
3. **Step 3:** Swap the pair that reduces cut the most, or have the least increase if no improvement.
4. **Step 4:** Repeat until each node is swapped once, keeping the best result.

Note: A heuristic; not guaranteed to find the global optimum.

4 Overlapping Communities

4.1 Clique Percolation Method (CPM)

Algorithm:

1. Identify all k -cliques (complete subgraphs of size k) in the graph.
2. Define adjacency between k -cliques: two k -cliques are adjacent if they share exactly $k - 1$ nodes.
3. Build a clique adjacency graph where nodes are k -cliques and edges connect adjacent k -cliques.
4. Each connected component of the clique adjacency graph constitutes one community.

Note: Allows overlapping communities, since a node may belong to multiple k -cliques.

4.2 Link Clustering

Algorithm:

1. Define a similarity measure between edges, e.g., Jaccard similarity of the neighbor sets of their shared endpoint:

$$S(e_{ik}, e_{jk}) = \frac{|n^+(i) \cap n^+(j)|}{|n^+(i) \cup n^+(j)|}$$

where $n^+(v)$ includes node v and its neighbors.

2. Initialize: each edge is its own cluster.
3. Compute the pairwise similarity matrix between edges.
4. Iteratively merge the pair of edge clusters with highest similarity.
5. Build the dendrogram of edge merges.
6. Cut the dendrogram at a chosen level to obtain communities.

Note: Clusters edges instead of nodes; a node belongs to as many communities as there are distinct clusters among its incident edges.

5 Network Models

5.1 Watts–Strogatz Model

Algorithm:

1. Create a ring lattice with n nodes; connect each node to its k nearest neighbors ($k/2$ on each side).
2. For each edge (u, v) in the lattice:
 - (a) With probability p : rewire the edge to a uniformly chosen random node, avoiding self-loops and duplicate edges.
 - (b) With probability $1 - p$: keep the edge unchanged.
3. Continue until all edges have been considered.

Key parameters:

$$\text{Number of edges} = \frac{nk}{2}, \quad n \gg k \gg \ln n \gg 1$$

Note: Produces small-world networks with high clustering ($p \approx 0$) and short average path lengths ($p \approx 1$). At intermediate p , both properties coexist.

5.2 Barabási–Albert (BA) Model

Algorithm:

1. Start with an initial connected graph of m_0 nodes.
2. At each time step:
 - (a) Add a new node with m edges ($m \leq m_0$).
 - (b) For each of the m new edges, connect to an existing node i with probability proportional to its degree:

$$\Pi(i) = \frac{k_i}{\sum_j k_j}$$

3. Repeat until the network reaches the desired size.

Mechanisms: Growth + Preferential Attachment.

Degree distribution:

$$P(k) \sim k^{-3}$$

5.3 Linearized Chord Diagram (LCD) Model

Purpose: Resolve sequential-attachment ambiguity in the BA model; provides a mathematically consistent formulation of preferential attachment.

Algorithm:

1. Start with an empty graph $G_1(0)$.
2. At time step t , add node v_t .
3. Add m links one at a time. For each new link, connect to node i with probability:

$$p(i = s) = \begin{cases} \frac{k_i}{2t-1} & \text{for existing nodes } s \\ \frac{1}{2t-1} & \text{for a self-loop on the new node} \end{cases}$$

4. Repeat until all m links for v_t are added, then proceed to $t + 1$.

Note: Ensures each link is attached independently; self-loops are later removed for final network analysis.

6 Origin of Preferential Attachment

6.1 Link Selection Model

Mechanism: A newly added node connects to a node chosen proportionally to its degree, without explicitly computing degree.

Algorithm:

1. Add a new node v .
2. Select a random edge uniformly at random from all existing edges.
3. Choose one endpoint of that edge uniformly at random.

4. Connect v to that endpoint.

Result: The probability of connecting to node i is proportional to k_i , since nodes with higher degree appear in more edges.

6.2 Copying Model

Motivation: Models how new web pages copy links from existing pages.

Algorithm:

1. Add a new node v .
2. Choose a node u uniformly at random.
3. With probability p : connect v directly to u .
4. With probability $1 - p$: choose a uniformly random neighbor of u and connect v to that neighbor.
5. Repeat for each of the m links to be added.

Note: Step 4 induces implicit preferential attachment because high-degree nodes appear more often as neighbors.

6.3 Optimization Model

Idea: Each new node minimizes a cost function that trades off distance to existing nodes against their importance (degree).

Cost function for connecting new node to node j via route through i :

$$C_i = \min_j [\delta \cdot d_{ij} + h_j]$$

where d_{ij} is the graph distance between i and j , h_j is a cost term inversely related to node importance, and δ controls the trade-off.

Outcomes depending on δ :

- Small δ : importance dominates $\Rightarrow$ star network.
- Intermediate δ : balance $\Rightarrow$ scale-free network.
- Large δ : distance dominates $\Rightarrow$ random-like network.

Distance d = how far the node is

Importance h = higher-degree nodes are “cheaper” to connect to

δ = trade-off between distance vs importance

6.4 Model A: Growth Only (No Preferential Attachment)

- New nodes attach to existing nodes uniformly at random.
- Degree distribution:

$$P(k) \sim e^{-k}$$

- Exponential tail; no hubs.

👉 As δ increases:

Star $\rightarrow$ Scale-free $\rightarrow$ Random-like

6.5 Model B: Preferential Attachment Only (No Growth)

- Fixed number of nodes N ; new edges added between existing nodes with probability proportional to degree.
- Results in dense, Gaussian-like degree distributions.
- Does not produce a stable power-law degree distribution.

Conclusion: Both growth and preferential attachment are necessary to produce scale-free networks.

7 Randomization / Null Models

7.1 Random Cuisine Strategy 1: Ingredient Randomization

Algorithm:

1. For each recipe in the dataset:
 - (a) Keep the number of ingredients fixed.
 - (b) Sample a new set of ingredients uniformly at random without replacement from the full ingredient vocabulary.
 - (c) Replace the original recipe with this randomized version.
2. Generate multiple independent randomized datasets to form the null ensemble.

7.2 Random Cuisine Strategy 2: Size-Preserving Randomization

Algorithm:

1. Compute the empirical recipe-size distribution from the original dataset.
2. Compute the cumulative distribution function (CDF) of recipe sizes.
3. For each synthetic recipe:
 - (a) Sample a recipe size from the empirical distribution (via inverse CDF sampling).
 - (b) Generate the recipe by sampling that many ingredients uniformly at random from the vocabulary.
4. Repeat to generate a full synthetic dataset.

8 Application Model

Bartoli's Algorithm

Goal:

Create a fake protein network that behaves like a real one (high clustering + short paths).

1. Create a chain: connect nodes in sequence $1 - 2 - 3 - \dots - N$
2. Pick two nodes i, j with probability biased by distance (closer nodes more likely)
3. Define neighborhoods:

$$A = \{i - 1, i, i + 1\}, B = \{j - 1, j, j + 1\}$$
4. Add edges between all nodes in A and all nodes in B (complete bipartite block)
5. Repeat until desired number of edges is reached

Notes:

- The backbone constraint (step 1) introduces structural regularity analogous to secondary structure.
- High clustering and short path lengths can emerge naturally from the block-assignment rule and backbone, without explicitly tuning these properties.

9 Configuration Model and Deg Preserving Ran

9.1 Degree-Preserving Randomization (Edge Rewiring)

Goal: Randomize the network while keeping the degree of every node unchanged.

Algorithm:

1. Randomly select two edges: (a, b) and (c, d) .
2. Rewire them to form (a, d) and (c, b) .
3. Ensure:
 - no self-loops
 - no duplicate edges

4. Repeat many times.

Idea: The number of connections per node stays the same, but the network structure becomes random.

Use:

- test whether clustering or communities are real or just due to degree distribution

9.2 Configuration Model

Goal: Generate a random network with a given degree sequence.

Algorithm:

1. Assign each node a number of stubs (half-edges) equal to its degree.
2. Ensure total number of stubs is even.
3. Randomly pick two stubs and connect them to form an edge.
4. Repeat until all stubs are paired.

Notes:

- Can produce self-loops and multi-edges.
- In large networks, these effects become negligible.

Connection probability:

$$p_{ij} = \frac{k_i k_j}{2L - 1}$$

: probability that nodes i and j connect is proportional to their degrees

→ high-degree nodes (hubs) are more likely to connect.

Idea: Nodes with higher degree are more likely to connect.

Drawbacks of BA Model:

BA model predicts $\gamma = 3$, but real networks have $\gamma \approx 2-5$

Generates undirected graphs, while many real networks are directed

Ignores real processes like link/node deletion and internal links

Treats all nodes the same (no intrinsic properties / fitness)

It's a simplified model, only captures basic mechanism (preferential attachment), not full real-world complexity

* Binomial to Poisson Approximation

Binomial Distribution

$$P(K) = \binom{N-1}{K} p^K (1-p)^{(N-1)-K}$$

for large N & small K i.e. $N \gg K$, the no. of ways to pick K nodes becomes:-

$$\binom{N-1}{K} \approx \frac{(N-1)^K}{K!} \quad \text{--- (1)}$$

$$\text{As } \langle K \rangle = p(N-1) \quad \therefore p = \frac{\langle K \rangle}{N-1}$$

$$p^K = \frac{\langle K \rangle^K}{(N-1)^K} \quad \text{--- (2)}$$

As $N \rightarrow \infty$, we have $\left(1 - \frac{n}{n}\right)^n \approx e^{-n}$

$$\left[1 - \frac{\langle K \rangle}{N-1}\right]^{N-1} \approx e^{-\langle K \rangle} \quad \text{--- (3)}$$

Multiplying (1), (2), (3)

$$P(K) \approx \left[\frac{(N-1)^K}{K!} \right] \cdot \left[\frac{\langle K \rangle^K}{(N-1)^K} \right] \cdot e^{-\langle K \rangle}$$

$$P(K) = e^{-\langle K \rangle} \frac{\langle K \rangle^K}{K!}$$